

MySQL for Data Engineering

Document ID	DOC004
Project	MySQL
Category	Database
Batch	batch_1

Purpose: Original educational notes created as a searchable PDF corpus for a portfolio-scale incremental RAG pipeline. The material is designed to contain meaningful technical sections that naturally produce multiple retrieval chunks.

Relational Foundations

MySQL stores structured data in tables made of rows and columns. Primary keys uniquely identify rows, while foreign keys represent relationships between tables. Normalization can reduce redundancy in transactional systems, while analytical models may intentionally denormalize data for simpler and faster queries.

Practical note. For reliable SQL pipelines, always confirm table grain and key behavior before using joins, aggregations, deduplication, or incremental extraction logic.

Joins

INNER JOIN returns matching rows from participating tables. LEFT JOIN preserves every row from the left table and adds matching data from the right table when available. Joins should use correct keys and data types. Duplicate keys on either side can multiply rows, so understanding table grain is essential before joining datasets.

Practical note. For reliable SQL pipelines, always confirm table grain and key behavior before using joins, aggregations, deduplication, or incremental extraction logic.

Aggregations and Window Functions

GROUP BY collapses rows into groups and is used with functions such as COUNT, SUM, AVG, MIN, and MAX. Window functions calculate values across related rows without collapsing the result. ROW_NUMBER, RANK, DENSE_RANK, LAG, LEAD, and aggregate window functions are useful for deduplication, ranking, comparisons with previous records, and analytical calculations.

Practical note. For reliable SQL pipelines, always confirm table grain and key behavior before using joins, aggregations, deduplication, or incremental extraction logic.

CTEs and Subqueries

A common table expression, introduced with WITH, names an intermediate query and can make complex SQL easier to read. Subqueries can appear in SELECT, FROM, WHERE, and HAVING contexts. Correlated subqueries reference the outer query and may execute logically for each outer row, so joins or window functions can sometimes be clearer or more efficient.

Practical note. For reliable SQL pipelines, always confirm table grain and key behavior before using joins, aggregations, deduplication, or incremental extraction logic.

Indexes and Performance

Indexes provide structures that can reduce the amount of data scanned for suitable filters and joins. They also consume storage and add overhead to inserts and updates. Query performance should be investigated using execution plans, selective predicates, appropriate indexes, correct join conditions, and avoidance of unnecessary columns or transformations.

Practical note. For reliable SQL pipelines, always confirm table grain and key behavior before using joins, aggregations, deduplication, or incremental extraction logic.

Incremental SQL Patterns

Incremental extraction often filters on an increasing identifier or modification timestamp. Upserts can insert new records and update existing ones based on a stable business or surrogate key. The pipeline should persist its successful boundary only after downstream processing succeeds, otherwise a failed run can incorrectly skip data.

Practical note. For reliable SQL pipelines, always confirm table grain and key behavior before using joins, aggregations, deduplication, or incremental extraction logic.

End of document.